CoachEdu Institute – Smart Coaching Management System

PROJECT REPORT

Project Title	CoachEdu Institute – Smart Coaching Management System
Version	1.0.0
Release Status	Final Release
Report Date	April 2026
Developed By	Ankita Swami
Live URL	https://coachedu-1-classes.vercel.app/
Repository	github.com/ankita1014-afk/coachEdu-1
Platform	Node.js + React.js + MongoDB
Deployment	Vercel + Render + Cloud Database

Full Stack Web Development Project | Academic Year 2025–2026

INDEX

Sr. No.	Main Heading	Page No.
1	Executive Summary	2
2	Introduction	3
3	Problem Statement & Scope	3
4	System Architecture & Technology Stack	4
5	Functional Specification & Modules	8
6	Database Engineering & Schema	9
7	Testing, Security & Deployment	11
8	COCOMO Cost Estimation & Conclusion	16

EXECUTIVE SUMMARY

The **CoachEdu Institute Management System** is a full-stack web application designed to digitize, automate, and centralize the operational workflow of coaching institutes. Traditionally, coaching institutes rely on manual systems for managing student records, attendance, homework, fees, and communication.

These methods are inefficient, error-prone, and lack scalability. This system provides a **role-based digital platform** for teachers, students, and parents. It enables centralized management of academic activities, real-time data access, and improved communication.

Built using modern technologies such as **React.js, Node.js, Express.js, and MongoDB**, the system ensures high performance, scalability, and smooth user experience.

Key Project Metrics at a Glance

METRIC	DETAIL	METRIC	DETAIL
Project Version	1.0.0 (Final Release)	**Status**	Production Deployed
Release Date	April 2026	**Architecture**	Client-Server
Frontend Runtime	React.js	**Backend Runtime**	Node.js v18 + /Express.js
Database	MongoDB Atlas	**Authentication**	Supabase Auth
Deployment	Vercel Edge Serverless	**Primary Users**	Teachers, Students, Parents

3. SYSTEM ARCHITECTURE & TECHNOLOGY STACK

3.1 Architectural Paradigm

The system follows a **Client-Server Architecture**, ensuring separation of concerns between frontend and backend.

Architectural Layers

LAYER	COMPONENTS	DESCRIPTION
CLIENT LAYER	React.js, HTML,CSS,JavaScript	The frontend is developed using React.js, providing a responsive and interactive user interface. It handles user interactions and communicates with the backend through API calls.
API GATEWAY LAYER	Express.js REST API | Node.js	The backend is built using Node.js and Express.js. It processes client requests, applies business logic, and communicates with the database.
BUSINESS LOGIC LAYER	Controllers: authController, studentController, teacherController, parentController, materialController, homeworkController, testController, attendanceController, notificationController	Modular controller layer executes domain-specific logic, validates input, and applies business rules such as student enrollment handling, homework assignment logic, test evaluation, and attendance tracking. It ensures proper data processing before interacting with the database.
DATA ACCESS LAYER	Mongoose ORM | MongoDB Driver	All database interactions are handled through Mongoose, which provides schema-based modeling and simplifies CRUD operations. It ensures structured queries, data validation, and efficient interaction with the MongoDB database.
DATABASE LAYER	MongoDB Atlas (Cloud Database) | Collections | Document-Based Storage	MongoDB Atlas provides a fully managed NoSQL database with collections such as students, parents, teachers, materials, homework, tests, attendance, and notifications. It supports scalable storage, fast querying, and flexible schema design.
AUTH LAYER	Custom Authentication System | Role-Based Access Control	The system implements a role-based authentication mechanism for students, teachers, and parents. It verifies user credentials during login and restricts access based on user roles, ensuring secure and controlled access to different modules.
DEPLOYMENT LAYER	Vercel (Frontend) | Render (Backend) | Global CDN | HTTPS	The frontend is deployed on Vercel, providing fast content delivery. The backend is hosted on Render, ensuring reliable API handling. The system supports secure communication using HTTPS and scalable deployment.

C.4 End-to-End Data Flow

The following table traces a complete request lifecycle through every layer of the system, from user interaction to database response:

#	EVENT	DATA FLOW
1	**User opens browser**	Browser loads static SPA from Vercel CDN (HTML/CSS/JS)
2	**Login / Registration**	SPA sends POST request (/api/auth/login or /api/auth/register) → backend validates credentials → user authenticated from database
3	**Session Handling**	User session maintained on client-side → role-based routing applied for dashboard access
4	**Dashboard Load**	SPA sends GET requests to backend APIs → server verifies user role → appropriate dashboard data prepared
5	**Data Fetch**	Controller queries MongoDB using Mongoose → retrieves student/teacher/parent data → returns JSON response
6	**Response Pipeline**	Backend sends JSON response → frontend parses data → UI updates dynamically
7	**Write Operations**	Teacher uploads materials/homework/tests → POST/PUT request → backend validates → data stored in MongoDB
8	**Student Interaction**	Student views materials, submits homework, attempts tests → requests sent → data saved/updated in database
9	**Parent Monitoring**	Parent accesses student performance → GET request → backend fetches data → displayed on dashboard
10	**Logout**	User logs out → session cleared on client-side → user redirected to login page

C.5 REST API Endpoint Architecture

The backend exposes multiple RESTful endpoints organized across different functional modules such as authentication, students, teachers, materials, homework, and tests. These endpoints handle client requests, process business logic, and interact with the database.

METHOD	ENDPOINT	ACCESS	DESCRIPTION
POST	/api/auth/register	Public	Register a new user (student/parent/teacher) with role assignment
POST	/api/auth/login	Public	Authenticate user credentials and allow system access
GET	/api/students	Teacher	Retrieve all student records.
GET	/api/students/:id	Teacher	Retrieve individual student details
POST	/api/students	Admin	Add a new student to the system
GET	/camps	All	List all upcoming and past camp events
DELETE	/api/students/:id	Admin	Remove a student record
POST	/api/teachers	Admin	Add new teacher.
GET	/api/teachers	Admin	Retrieve all teacher records
POST	/api/materials	Teacher	Upload new study material
GET	/api/materials	All	Fetch all study materials
GET	/api/homework	Teacher/Student	Retrieve homework assignments
POST	/api/homework/submit	Student	Submit homework
GET	/api/tests	Teacher/Student	Fetch test details
POST	/api/tests	Teacher	Create new test

C.6 Security Architecture

Security is implemented across different layers of the system to ensure data protection, controlled access, and safe communication between client and server.

Network Layer: The system uses HTTPS protocol for secure communication between client and server, ensuring encrypted data transfer.

- **API Gateway Layer:** CORS policies restrict API access to authorized frontend origins only, preventing unauthorized cross-origin requests.
- **Authentication Layer:** User authentication is handled using login credentials (student ID / teacher ID and password), validated against the database.
- **Authorization Layer:** Role-based access control (Admin, Teacher, Student, Parent) ensures that users can only access features permitted to their role.
- **Data Layer:** MongoDB ensures secure data storage, and database queries are controlled via backend logic to prevent unauthorized access.
- **Input Validation Layer:** Server-side validation checks all inputs (forms, login, submissions) to prevent invalid data, injection attacks, and errors.

3.2 Complete Technology Stack

LAYER	TECHNOLOGY	PURPOSE
Frontend	React.js, HTML5, CSS3, JavaScript	Interactive user interface and dynamic rendering
Backend	Node.js, Express.js Framework	API handling and business logic execution
Database	MongoDB (Atlas)	NoSQL database for storing application data
Auth	Session-based / Credential-based Login	User authentication and access control
DevOps	Git, Vercel (Frontend), Render (Backend)	Deployment and version control
Security	CORS, Input Validation, Role-based Access	Secure API communication and data protection

5.3 Key Design Decisions

- **Unique Identifiers:**
  Each entity uses unique identifiers (such as studentId, teacherId) to ensure distinct records and easy referencing across collections.
- **Reference-Based Relationships:**
  Collections are linked using identifiers (e.g., studentId in fees, attendance, and homework), maintaining logical relationships between entities.
- **Flexible Schema Design:**
  MongoDB allows dynamic fields such as submissions, questions, and recipients arrays for handling complex data structures.
- **Indexed Fields:**
  Frequently queried fields like studentId, teacherId, and class are indexed to improve query performance.
- **Data Consistency:**
  Validation checks are applied at the application level to ensure correct and consistent data entry.

6. SECURITY & COMPLIANCE

Security is implemented across multiple layers of the system to ensure safe data handling, controlled access, and reliable system performance.

SECURITY CONTROL	IMPLEMENTATION	PROTECTION
CORS Policy	Configured in Express.js to allow only authorized frontend requests	Prevents unauthorized cross-origin access
Input Sanitization	Server-side validation for all user inputs and forms	Prevents invalid data and injection attacks
Authentication	Credential-based login system for all use	Ensures only authorized users access the system
Role-Based Access Control	Role-based restrictions for Admin, Teacher, Student, Parent	Prevents unauthorized feature access
Environment Variables	Sensitive data stored in .env files	Protects credentials and configuration data
HTTPS Enforcement	Secure communication using HTTPS protocol.	Prevents data interception and attacks

- **403 Forbidden:** Access denied based on role permissions.

8. DEPLOYMENT STRATEGY

The system follows a cloud-based deployment strategy ensuring scalability, accessibility, and performance optimization.

COMPONENT	STRATEGY	BENEFIT
Backend API	Node.js + Express deployed on Render	Scalable backend with easy deployment
Frontend Assets	React app deployed on Vercel	Fast global content delivery
Database	MongoDB Atlas (Cloud Hosted)	Reliable and scalable database
Environment Variables	Stored in .env files	Secure handling of sensitive data
Local Development	VS Code + Node.js environment	Efficient development workflow

8.1 Live Deployment URLs

Live Application	**http://localhost:3000**
Source Repository	github.com/ankita1014-afk/coachEdu-1

9. FUTURE ROADMAP & SCALABILITY

To enhance the system and support larger-scale usage, the following improvements are planned:

#	FEATURE	DESCRIPTION	PRIORITY
1	Email Notifications	Send automated alerts for homework, tests, and announcements	High
2	Report Generation	Export student performance reports in PDF/Excel format	High
3	Mobile Application	Develop mobile app for better accessibility	Medium
4	Advanced Analytics	Track student performance trends and insights	Long-term

B. COCOMO PROJECT COSTING

B.1 Overview of COCOMO Model

The Constructive Cost Model (COCOMO), developed by Barry Boehm, is used to estimate software development effort, duration, and cost based on the size of the software measured in Kilo Lines of Code (KLOC)
This project uses the **Basic COCOMO model in Semi-Detached mode**, as it represents a medium-scale application with moderate complexity, involving both frontend and backend development handled by a single developer.

The three COCOMO modes and their applicability:

- **Organic Mode:** Small teams, familiar technology, simple requirements. (< 5 KLOC)
- **Semi-Detached Mode:** Medium teams, mix of experienced and novice developers, mixed requirements. (5–30 KLOC) — Selected for this project.
- **Embedded Mode:** Tight constraints, complex real-time systems, large teams. (> 30 KLOC)

B.2 COCOMO Input Parameters

PARAMETER	VALUE	JUSTIFICATION
Project Type	Semi-Detached	Moderate size; team familiar with Node.js & Supabase
Size Metric	KLOC (Kilo Lines of Code)	Source lines across frontend + backend modules
Estimated KLOC	8.5 KLOC	JS SPA ~4.5K + Express API ~3.5K + SQL ~0.5K
COCOMO Mode	Basic COCOMO	Suitable for academic project cost estimation
Mode Coefficients	a=3.0, b=1.12 (Semi-Detached)	Standard Boehm coefficients for semi-detached mode

2. PROJECT SIZE ESTIMATION (KLOC)

2.1 Module-wise Lines of Code Breakdown

The project codebase is divided across frontend SPA modules, backend API controllers, and database SQL scripts. Estimated Source Lines of Code (SLOC) per module:

#	MODULE	COMPONENT	EST. SLOC	NOTES
1	Auth Module (Login/Register UI + API)	Frontend + Backend	280	Authentication logic, validation
2	Teacher Dashboard	Frontend	220	UI rendering, data display
3	Student Dashboard	Frontend + API	180	Profile & data fetch
4	Subjects Module	Frontend + Backend	260	CRUD operations
5	Materials & Homework	Frontend + Backend	200	Upload & access logic
6	Test/Quiz Module	Frontend + Backend	210	Question & result handling
7	Notifications System	Frontend + Backend	160	Announcement logic
8	Database Schema (MongoDB)	Database	120	Collections, indexing
9	Deployment Config	DevOps	70	Hosting & environment setup

Including comments, configuration files, and supporting scripts, the effective project size is rounded to approximately **2.0 KLOC**, which is used as the input for COCOMO estimation.

3. BASIC COCOMO CALCULATIONS

3.1 Formulae (Semi-Detached Mode)

METRIC	FORMULA	DESCRIPTION
Effort (PM)	PM = 3.0 × (KLOC)^1.12	Person-Months required to complete the project
Development Time	TDEV = 2.5 × (PM)^0.38	Total time required for development in months
Team Size	N = PM / TDEV	Average number of developers required
Productivity	P = KLOC / PM	Productivity measured in KLOC per person-month

3.2 Step-by-Step Calculation

STEP 1 — Effort Estimation

PM = 3.0 × (KLOC)^1.12

PM = 3.0 × (2.0)^1.12

PM = 3.0 × 2.17

PM = 6.51 ≈ 6.5 Person-Months

STEP 2 — Development Time

TDev = 2.5 × (PM)^0.38

TDev = 2.5 × (6.5)^0.38

TDev = 2.5 × 1.87

TDev = 4.68 months ≈ 4.7 months (within 4–5 month target ✓)

STEP 3 — Team Size

N = PM / TDev = 6.5 / 4.7 = 1.38 ≈ 1 Developer (confirmed ✓)

STEP 4 — Productivity

P = KLOC / PM = 2.0 / 6.5 = 0.31 KLOC per Person-Month

3.3 Results Summary

COCOMO METRIC	VALUE	INTERPRETATION
Input — Project Size (KLOC)	2.0 KLOC	~1,700 SLOC + configs & comments
Input — COCOMO Mode	Semi-Detached	Moderate complexity full-stack project
Effort Required	**6.5 PM**	Total development effort required
Development Time	**4.7 months**	Matches academic timeline ✓
Team Size	**1 Developer**	Suitable for individual project
Productivity	**0.31 KLOC/PM**	Efficient development rate

4. PROJECT COST ESTIMATION

4.1 Monthly Rate Derivation

With an estimated development effort of approximately **4.7 months** (single developer), the project cost is calculated based on a standard academic development rate.

Maximum Rate = Total Budget / Duration = ₹35,000 / 4.7 months = ₹7,446 / month

Developer Rate Used = ₹7,000 / month

A rate of ₹7,000/month is a realistic student developer/freelance intern rate for a Tier-2 Indian city (Nashik). This rate is deliberately set below the maximum to preserve budget headroom for documentation, testing, and contingency reserves.

4.2 Detailed Cost Breakdown

#	COST ITEM	BASIS / CALCULATION	AMOUNT (₹)
1	**Developer Labour (Solo)**	4.7 PM × ₹7,000/month	₹35,900
	→ Requirements & Design (0.5 PM)	0.5 × ₹7,000	₹3,500
	→ Frontend Development (1.5 PM)	1.5 × ₹7,000	₹10,500
	→ Backend API Development (1.5 PM)	1.5 × ₹7,000	₹10,500
	→ Database Engineering (0.5 PM)	0.5 × ₹7,000	₹3,500
	→ Testing, Deployment & Docs (1.0 PM)	0.7 × ₹7,000	₹4,900
2	**Database (MongoDB Atlas)**	Free Tier — no cost incurred	₹0
3	**Frontend/Backend Hosting**	Vercel/Local/Render	₹0
4	**Domain / Hosting**	Free subdomain	₹0

5	**GitHub Repository (Version Control)**	Free public repository	₹0
6	**Docker Desktop (Local Dev)**	Free Community Edition	₹0
TOTAL PROJECT COST		Labour (4.7 PM × ₹7,000) + ₹0 Infrastructure	**₹32,900**

Total Project Cost = ₹32,900 | Budget Utilization: 100% | Status: WITHIN BUDGET ✓

Infrastructure cost = ₹0 (all free tiers) | 100% of budget allocated to developer effort

5. EFFORT DISTRIBUTION & SCHEDULE

5.1 Phase-wise Effort Allocation (4.7 Person-Months)

#	PHASE	DURATION	EFFORT (PM)	COST (₹)	% TOTAL
1	Requirements & System Design	Weeks 1–2	0.5	₹3,500	11%
2	Database Engineering (Schema + Models)	Weeks 3–4	0.5	₹3,500	11%
3	Backend API Development	Weeks 5–8	1.5	₹10,500	32%
4	Frontend Development (React UI)	Weeks 9–12	1.5	₹10,500	32%
5	Integration, Testing & QA	Weeks 13–14	0.4	₹2,800	9%
6	Deployment & Documentation	Weeks 15–16	0.3	₹2,100	5%
TOTAL	**Full Project**	**16 Weeks / ~4.7 Months**	**4.7 PM**	**₹32,900**	**100%**

5.2 18-Week Developer Timeline

WEEK(S)	MONTH	TASK
1	Month 1	Project setup: Git repository initialization, Docker configuration, backend project initialization, environment setup.
2	Month 1	Requirements finalization, ER diagram design, API contract definition, SPA wireframe design.
3–4	Month 1	Database schema creation, PostgreSQL tables, indexing, foreign key constraints, and security policies.
5–6	Month 2	Backend authentication module implementation, role-based access control, login and registration endpoints.
7–8	Month 2	Backend development: cadet management, camp management APIs, and enrollment workflows.
9–10	Month 3	Backend modules: attendance tracking, achievement workflow management, and study materials APIs.
11–12	Month 3	Frontend foundation: SPA structure, authentication UI, session handling, role-based routing.
13–14	Month 4	Frontend dashboard: officer analytics panel, camp management UI, attendance interface.
15–16	Month 4	Frontend cadet portal: achievements submission UI, study materials access, profile management.
17	Month 5	Integration testing, API validation, multi-role testing, and security checks.
18	Month 5	Final deployment, CDN configuration, environment setup, and documentation completion.

6. FINAL COCOMO SUMMARY

PARAMETER	VALUE	STATUS
COCOMO Mode	Organic	Appropriate for solo academic project
Project Size (KLOC)	2.0 KLOC	~1,700 net SLOC + overhead
Estimated Effort	**4.7 Person-Months**	Boehm Organic formula applied
Development Time	**4.54 months**	Within 4–5 month target ✓
Team Size Required	**1 Developer**	Suitable for solo development ✓
Productivity	0.43 KLOC/PM	Efficient output for academic project
Developer Rate	₹7,000 / month	Student-level cost estimation
Total Project Cost	**₹32,900**	Within defined budget ✓
Infrastructure Cost	₹0	Free-tier services utilized

Budget Utilization	**100%**	Entire budget allocated to development effort.